

Lecture 5 • Mon 10/12/2026

CS160 Compilers — Fall 2026

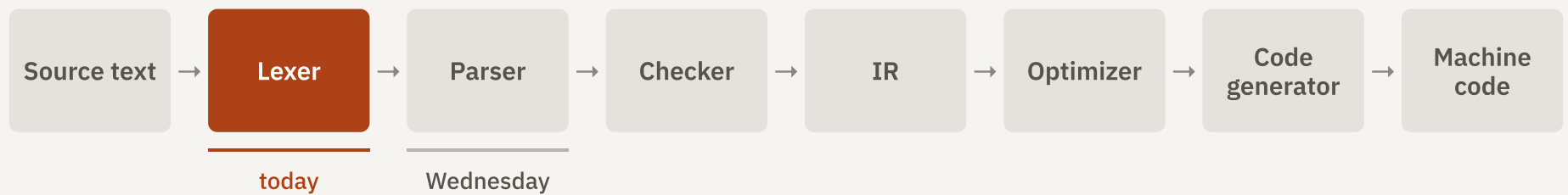
Lexing.

Yu Feng

Think it through before I say anything. PA2 is out — this lecture and the next two *are* PA2.

01

Where we are.

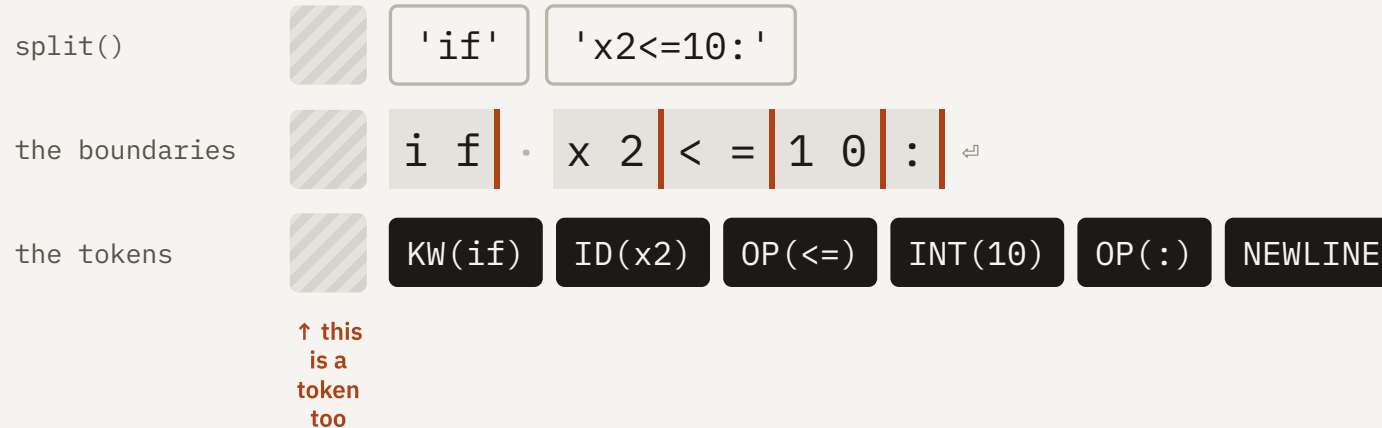


So far ast.parse did the first two arrows for free.

PA2: you write them.

Today the first arrow — the easiest part of a compiler, with the most theory.

It looked like `split()`. It isn't.



`x2` is one word, not two. `<=` is one token, not two — and no space tells you where either one ends.

`if` looks exactly like a variable name; nothing in the characters says otherwise.

And in Python the *indentation* carries meaning that isn't any character you can see.

The real job: an infinite set of strings has to become a short, finite list of *kinds* — and everything downstream works only because that list is finite.

How do you turn a stream of characters into the words of a program?

By the end you can turn a regular expression into an NFA and that NFA into a DFA, on paper, and say how many states you get.

You can run a DFA across an input string and mark where maximal munch ends each token.

You can write out the indent stack and the INDENT/DEDENT tokens a program produces, line by line.

All three are pencil-and-paper — they are **Midterm 1**. PA2 is the same three in code, and it's out today.

A token is a *kind* plus a *lexeme* plus a *position*.

```
source:    while i < n:
tokens:    KW(while)  ID(i)  OP(<)  ID(n)  OP(:)  NEWLINE
           3:1        3:7    3:9    3:11  3:12    3:13
```

kind

which of the eight classes this is — KW, ID, OP, ...

lexeme

the exact characters the token was made of: while, i, <

position

line and column of its first character.

The lexer drops spaces and comments and keeps what the parser will ask about: what kind, which characters (the *lexeme*), where.

Eight kinds. The whole lexical language of ChocoPy through PA3.

```
# ChocoPy v2.2 §3.3 – the keyword list is exactly Python 3's
KEYWORDS = {"False", "None", "True", "and", "as", "assert", "async", "await",
            "break", "class", "continue", "def", "del", "elif", "else",
            "except", "finally", "for", "from", "global", "if", "import",
            "in", "is", "lambda", "nonlocal", "not", "or", "pass", "raise",
            "return", "try", "while", "with", "yield"}

# ChocoPy v2.2 §3.5 – operators and delimiters; multi-character ones first
OPERATORS = ["==", "!=", "<=", ">=", "//", "->",
            "+", "-", "*", "%", "<", ">", "=",
            "(", ")", "[", "]", ",", ":", "."]

# plus: ID [A-Za-z_][A-Za-z0-9_]* §3.2      INT 0|[1-9][0-9]* §3.4.2
# plus: NEWLINE INDENT DEDENT EOF      <- the ones Python needs and C doesn't
```

We use nine of the 35 keywords today — `if` `elif` `else` `while` `True` `False` `not` `and` `or` — and `def` `return` `pass` `None` `class` as our subset grows. The other twenty-odd are **reserved**: the lexer still emits KW for them, so they can never be identifiers.

One thing to say out loud: `print` is an ordinary ID, exactly as in Python 3, *not* a keyword — §3.3 has no `print` in it.

Strings arrive in PA4 — §3.4.1: a sequence of ASCII characters 32–126 delimited by, and including, double quotes, with the escapes `\n` `\t` `\\`.

Every token kind is a regular expression.

regular expression a pattern that names a *set* of strings. $[0-9]^+$ is not one number; it is every number.

ID	<code>[A-Za-z_][A-Za-z0-9_]*</code>	§3.2
INT	<code>0 [1-9][0-9]*</code>	§3.4.2 <code>max 2**31 - 1</code>
OP	an alternation of 20 literals	§3.5

§3.2 — an identifier is “a contiguous sequence of characters containing the uppercase and lowercase letters A through Z, the underscore and, except for the first character, the digits 0 through 9.”

§3.4.2 — an integer literal is “a sequence of one or more digits 0-9, where the leftmost digit may only be 0 if it is the only character in the sequence.”

So `007` is not one `INT`, and the ceiling of $2^{31} - 1$ is exactly the range of ChocoPy's `int`, which is 32-bit signed, $-2^{31} \dots 2^{31}-1$ (§2.5.1) — a larger literal is a *lexical* error, not an overflow at run time.

The lexer's job: at the cursor, which pattern matches, and how far?

We need a *machine* that answers that fast.

That machine is a **DFA** — a *deterministic finite automaton*: finitely many states, one character read at a time, and from each state exactly one arrow to follow. The rest of this segment builds it.

Three operators. Everything else is sugar.

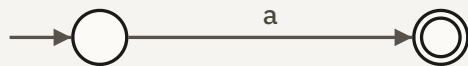
<code>a</code>	the character <code>a</code>	
<code>r1 r2</code>	<code>r1</code> then <code>r2</code>	(concatenation)
<code>r1 r2</code>	<code>r1</code> or <code>r2</code>	(alternation)
<code>r*</code>	zero or more <code>r</code>	(Kleene star)
sugar:	<code>r+ = r r*</code>	<code>r? = (r ϵ)</code> <code>[a-c] = (a b c)</code>

ϵ the empty string — the pattern that matches *no characters at all*. So `r?` reads “`r`, or nothing”.

You'll build machines for the three real operators. The sugar is free.

Regex $\rightarrow$ NFA, one rule per operator.

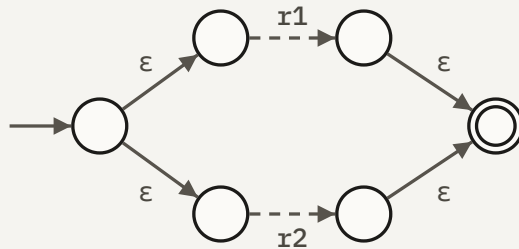
a



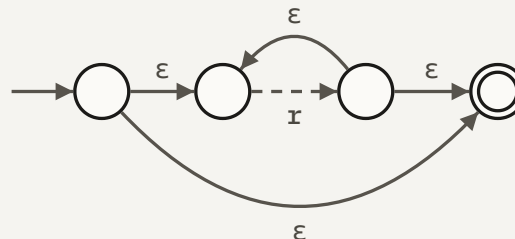
r1 r2



r1 | r2



r*



NFA

nondeterministic finite automaton — a machine that may be in **many states at once**, where a DFA is in exactly one. The lexer wants one. Formal definition on slide 11, next to the DFA's.

ϵ -transition

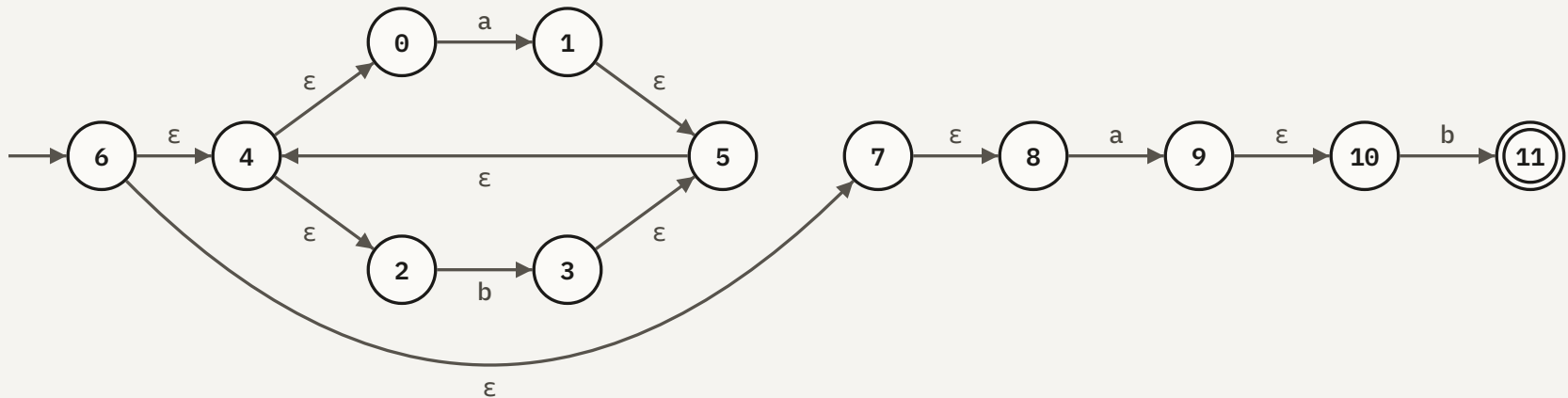
an edge the machine may follow **without reading a character**. On slide 8 ϵ was the empty string; an edge labelled with it consumes nothing, so it is a free move.

----- a whole fragment, start to accept

Every fragment has one start and one accept, and fragments connect only by ϵ edges. That's why the rules compose.

Worked: $(a|b)^*ab$

nfa2dfa.py '(a|b)*ab' · 12 states · start 6, accept 11



12 states for a 7-character regex. Thompson is wasteful on purpose: simple rules, no thinking. The next step cleans up.

Both are five things. They differ in one.

	NFA = (Q, Σ, δ, q0, F)	DFA = (Q, Σ, δ, q0, F)
Q	a finite set of states	(same)
Σ	the alphabet: the input's characters	(same)
q0	the start state, q0 in Q	(same)
F	the accepting states, F subset of Q	(same)
δ	$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$ a SET of next states: maybe none, maybe several, maybe on ε (read nothing) never on ε	$\delta : Q \times \Sigma \rightarrow Q$ exactly ONE next state,
	accepts a string if SOME path from q0 reading it ends in F	accepts if THE path from q0 reading it ends in F

Four rows identical; the whole difference is δ . In the DFA it is a *function* — one (state, character), one answer — and that is what *deterministic* means: no choices, no backtracking, one step per character. That is why the lexer wants one.

The next slide builds the DFA's five parts out of the NFA's five parts. Read its pseudocode as filling in the right-hand column, one line per row.

NFA $\rightarrow$ DFA: a DFA state is a *set* of NFA states.

subset construction

the algorithm's name, and its whole idea: one DFA state *is* a subset of the NFA's states — the answer to “where could the NFA be right now?” The NFA is in many states at once, the DFA in exactly one, so the DFA's one state has to be the NFA's many.

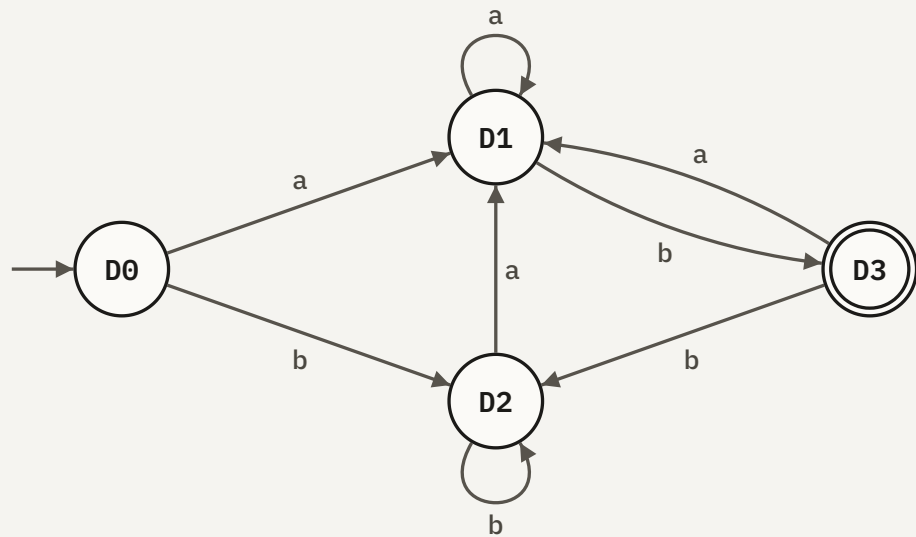
```
D0 =  $\epsilon$ -closure({start})                                # q0
for each DFA state S and character c:                      #  $\Sigma$ : same alphabet as the NFA
    T =  $\epsilon$ -closure( move(S, c) )                      # reachable on c, then  $\epsilon^*$        $\delta(S, c) = T$ 
    if T is new, add it; add edge S --c--> T              # Q: the sets reached
accepting = every S containing the NFA's accept state     # F
```

Each line fills one slot of slide 11's tuple: Q is the subsets we reach, $\delta(S, c) = \epsilon\text{-closure}(\text{move}(S, c))$ is one set per (S, c) — a function, so the result is deterministic by construction — $q_0 = D_0$, F = the subsets holding an NFA accept state.

Both primitives are the NFA's δ applied to a set: $\text{move}(S, c) =$ the union of $\delta(q, c)$ over q in S , the states one c edge away; $\epsilon\text{-closure}(S) = S$ plus everything reachable from it along $\delta(\cdot, \epsilon)$ edges, repeated until nothing new appears. Repeat the loop until no new sets — finitely many subsets of Q , so it terminates.

Worked: the DFA for $(a|b)^*ab$

D0 = {0,2,4,6,7,8}	D0 --a--> D1	D0 --b--> D2
D1 = {0,1,2,4,5,7,8,9,10}	D1 --a--> D1	D1 --b--> D3
D2 = {0,2,3,4,5,7,8}	D2 --a--> D1	D2 --b--> D2
D3 = {0,2,3,4,5,7,8,11} ACCEPT	D3 --a--> D1	D3 --b--> D2



Four states.

D1 = “last character was a”; **D3** = “last two were ab”.

The machine remembers exactly what it needs.

Some DFA states are twins.

Two states are *equivalent* if no input can tell them apart from here on.

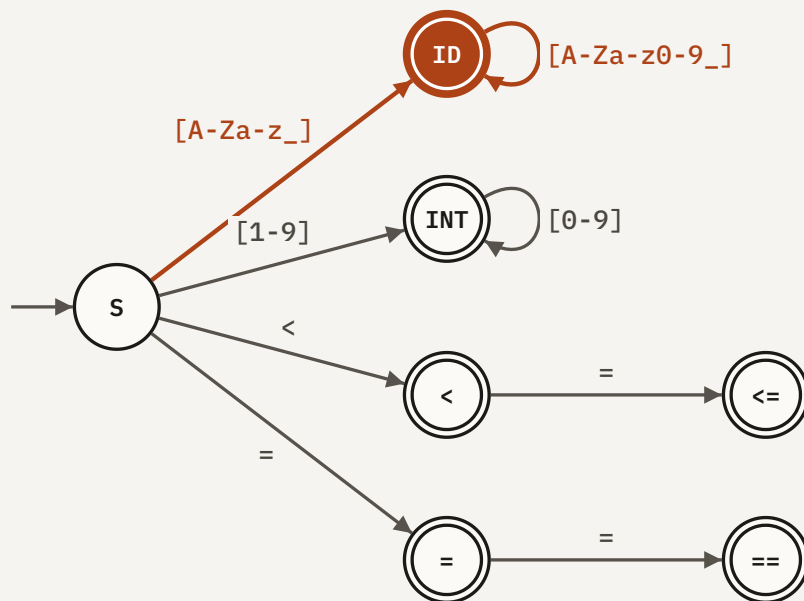
Start with two groups — accepting and not — and split a group whenever two members go to different groups on some character; stop when nothing splits.

For $(a|b)^*ab$ nothing splits: 4 is minimal. **Exam: I'll ask you to merge, not to prove.**

Longest match wins.

maximal munch the rule's name: at each position take the *longest* string any pattern matches — never the first pattern that happens to match.

ID, INT, <, <=, =, == merged into one DFA (a subset)



Run the DFA as far as it goes; remember the last accepting position; emit that token, reset, continue.

<= is one token *because* the DFA kept going.

Puzzle bug #2: Python's `re` alternation stops at the first alternative that matches. It does not munch.

if matches the identifier pattern. So does **iffy**.

lexer.py

```
Token("KW" if word in KEYWORDS else "ID", word, line, i + 1)
```

Lex it as an identifier; then look it up. Every real lexer does this. A separate `if` regex splits `iffy` into `if` + `fy` unless you munch — and then you still need a tie rule.

Two ways to get the DFA.

Hand-written (ours)

a loop of `if c.isdigit() ...` the DFA is implicit in the branches.

Generated

patterns in, DFA out (flex, ocamllex, Python's `re`).

PA2 allows either. Hand-written: ~80 lines, readable errors. Generated: 20 lines, and you must know your tool's munch rule.

What the LLM's lexer actually did.

```
$ python3 puzzle.py
x2 = 7      [('ID', 'x'), ('INT', '2'), ('OP', '='), ('INT', '7')]
if x <= 10:  [('ID', 'if'), ('ID', 'x'), ('OP', '<'), ('OP', '='), ('INT', '10'), ('OP', ':')]
```

Three bugs, each a slide you just saw: no digits in **ID** (7); first-match not longest-match (16); no keyword lookup (17). Fix for bug 2 in `re`: list `==`, `<=` *before* `=`, `<` — our OPERATORS list does exactly that.

C has braces. Python has whitespace.

```
while i < n:           # NEWLINE
    if i % 2 == 0:      # INDENT ... NEWLINE
        print(i)        # INDENT ... NEWLINE
    else:              # DEDENT ... NEWLINE
        print(-i)       # INDENT ... NEWLINE
    i = i + 1          # DEDENT ... NEWLINE
print(i)              # DEDENT ... NEWLINE (loop.py, lines 3-9)
```

The parser wants braces. The lexer manufactures them: `INDENT = {`, `DEDENT = }`, `NEWLINE = ;`.

§3.1 fixes what a line *is*: a **logical line** contains at least one token that is not whitespace or a comment, and ends with a `NEWLINE` token. A physical line holding only spaces, tabs and possibly a comment is ignored and generates no `NEWLINE`. A comment starts at a `#` that is not inside a string literal and runs to the end of the physical line.

The lexer's only memory is a stack of widths.

```
3 while i < n:
4     if i % 2 == 0:
5         print(i)
6     else:
7         print(-i)
8     i = i + 1
9 print(i)
```



indent stack

indent stack

a stack holding the indentation width of every block currently open; its top is the width of the block we are inside.

A single zero is pushed on the stack before the first line and is never popped; the numbers on the stack are always strictly increasing.

Width **equal** to the top: nothing happens.

Width **larger** than the top: push it, generate one INDENT.

Width **smaller**: it must already be on the stack — pop every larger number and generate one DEDENT per pop; if it is not on the stack, that is an error.

At end of input, generate one DEDENT for each number remaining on the stack that is larger than zero.

tab expansion §3.1.5, quoting the Python 3 reference: “Tabs are replaced (from left to right) by one to eight spaces such that the total number of characters up to and including the replacement is a multiple of eight.”

The indent stack, traced.

line	source	stack after	emits
3	while i < n:	[0]	—
4	if i % 2 == 0:		
5	print(i)		
6	else:		
7	print(-i)		
8	i = i + 1		
9	print(i)		
	EOF		

0

indent stack

Line 6: 8→4, one pop. Line 9: 4→0, one pop. At EOF, pop everything left — a program ending inside a loop still needs its closing braces.

The code.

lexer.py · the loop body

```
def indent_width(raw):                # §3.1.5: tab -> next multiple of 8
    w = 0
    for ch in raw:
        if ch == " ": w += 1
        elif ch == "\t": w += 8 - (w % 8)
        else: break
    return w

indents = [0]                        # the zero is pushed once, never popped
for line_no, raw in enumerate(src.split("\n"), start=1):
    stripped = raw.lstrip(" \t")
    if stripped == "" or stripped.startswith("#"):    # not a logical line (§3.1)
        continue
    width = indent_width(raw)
    if width > indents[-1]:
        indents.append(width); tokens.append(Token("INDENT", "", line_no, 1))
    else:
        while width < indents[-1]:
            indents.pop(); tokens.append(Token("DEDENT", "", line_no, 1))
            if width != indents[-1]: raise LexError(f"line {line_no}: unindent ...")
    lex_line(stripped, line_no, tokens)
    tokens.append(Token("NEWLINE", "", line_no, len(raw) + 1))
while indents[-1] > 0:                # end of input: one DEDENT per open block
    indents.pop(); tokens.append(Token("DEDENT", "", line_no, 1))
```

Blank and comment lines are skipped *before* the width check — else an empty line would dedent to 0, and §3.1 says such a line is not a logical line at all.

`indent_width` is the only part students skip. A tab is not one character of indentation and not eight either — it is however many take you to the next multiple of eight.

The lexer on a real program.

loop.py · lines 3-9

```
3 while i < n:
4     if i % 2 == 0:
5         print(i)
6     else:
7         print(-i)
8     i = i + 1
9 print(i)
```

\$ python3 lexer.py loop.py · lines 3-9 of the output

```
3:1  KW      while
3:7  ID      i
3:9  OP      <
3:11 ID      n
3:12 OP      :
3:13 NEWLINE
4:1  INDENT
4:1  KW      if
...
6:1  DEDENT
6:1  KW      else
...
9:1  DEDENT
9:1  KW      print
```

About 90 lines. `python3 -m tokenize loop.py` gives the same tokens in the same order. That comparison is your PA2 lexer test.

Live · about a minute

Break it.

bad3.py · line 3 is indented 2 spaces under a 4-space block

```
$ python3 lexer.py bad3.py  
LexError: line 3: unindent does not match any outer level
```

Python's exact message. The while loop popped past 2 and never found it.

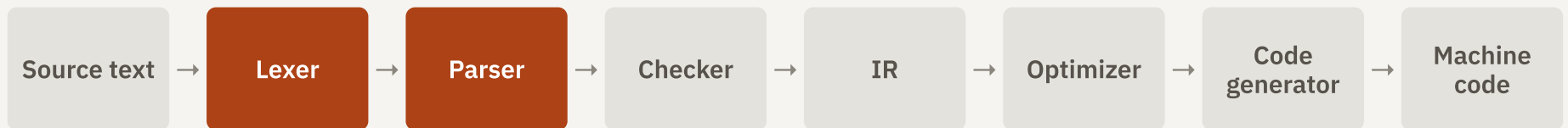
Every token carries line and column. Never lose them.

```
@dataclass
class Token: kind: str; text: str; line: int; col: int
```

```
$ python3 lexer.py bad2.py
LexError: line 1, col 5: unexpected character '$'
```

Wednesday's parser errors and week 5's type errors all say `line L, col C` — and that information exists only here. Thread it through the token. **PA2 grades error *positions*.**

PA2 — out today, due Mon 10/19.



Lexer (today) + parser (Wed, Mon) for PA1's subset of ChocoPy, producing Python ast nodes.

Oracle: `ast.dump(yours) == ast.dump(ast.parse(src))`. Your lecture-3 compiler runs unchanged on your tree.

Section Thursday: PA2 lab, bring your lexer.

Wednesday: tokens → tree.

Grammars, ambiguity, why **1 - 2 - 3** is two trees and only one is right.

Recursive descent: one function per rule — a working expression parser by the end of class.

Read EaC 3.1–3.3.